Programski prevodioci

02 Sintaksna analiza

25-26

Dunja Vrbaški

Fakultet tehničkih nauka, Novi Sad

Sintaksna analiza - Parser

** parse tree, AST*

- Faza prevođenja: proveriti da li je ulazni niz tokena formiran na osnovu pravila po kom se grade konstrukcije u PL

Šta bi bila greška u sintaksnoj analizi?

Šta bi bila greška u sintaksnoj analizi?

```
if x == 5
{
   y 3 //komentar
}
```

Zavisi od toga kako definišemo!

- Možda uslov mora da ima zagrade.
- Možda nijedna struktura/konstrukcija u našem jeziku nema oblik: <identifikator> <broj>.
- Možda svaki iskaz mora imati “;” na kraju

...

Razmatranje tipa promenljive x (da li je x uopšte broj) i da li sme da se vrši ovakvo upoređivanje pripada sledećoj fazi (type checking; semantička analiza)

Nekoliko odluka

- Koje tokene ćemo prepoznavati?
- Koji tokeni imaju vrednosti?

- Kako skener i parser komuniciraju, kako se preuzimaju tokeni?

- Kako definisati pravila sintakse?

- Kako implementirati parser (sintaksni analizator)?
- Koje strukture ćemo koristiti?

- ...

Kako implementirati parser?

Kako implementirati parser?

- čitamo tokene sa ulaza
- nekako proveravamo da li su raspoređeni pravilno
- informišemo o rezultatu (ok // greška)

Kako implementirati parser?

- čitamo tokene sa ulaza
- nekako proveravamo da li su raspoređeni pravilno
- informišemo o rezultatu (ok // greška)

- ručno, proizvoljno
- ručno + formalizacija pravila
- alat + formalizacija pravila

Leksika programskih jezika predstavlja regularan jezik.
Velik deo sintakse programskih jezika predstavlja kontekstno slobodan jezik.

Bez obzira na pristup, pojavljuju se određeni problemi.

Na primer: dvosmislenost, konflikt, prioritet operacija, obrada grešaka...

Takođe, postavlja se pitanje šta je ulaz za sledeću fazu (semantička analiza)?

- ništa – radi se paralelno, odmah se utvrđuje da li je i sintaksno i semantički sve u redu
- realizuje se neka interna struktura na osnovu koje se može lako utvrditi da li važe semantička pravila

Pokušaćemo formalno da opišemo pravila strukture koda napisanog u PL

```
naredbe = if_naredba ili for_naredba ili naredba_dodele
if_naredba = if (uslov) naredbe else naredbe
...
```

Skeniranje – konstruisanje reči od karaktera
Parsiranje – konstruisanje rečenica od reči

stringovi *azbuka*

Teorija formalnih jezika
Teorija automata

Kasnije...

BNF (Backus-Naur) forma

pravila sintakse PL se često navode u BNF formi

```
<assignment statement> ::= <variable> = <expression>
<expression> ::= <term>
<expression> ::= <expresion> + <term>
<term> ::= …
```

- terminali (simboli) i neterminali (pojmovi)
- neterminali (pojmovi) se navode koristeći “<” i “>”
- “::=” oznaka za pravilo izvođenja

Zapisi budu i drugačiji:

- izostavlja se <>, ali se nekako pravi razlika između pojmova i simbola
- koristi se →
- koristi se :=
- | se koristi za alternative (umesto navođenja posebnih pravila)
- …

<assignment statement> ::= <variable> = <expression>
<expression> ::= <term>
<expression> ::= <expresion> + <term>
<term> ::= …

assignment statement → variable = expression
expression → term | expression + term
term → …

```
time   → hour ":" minute
       | hour ":" minute ":" second

hour   → 2DIGIT

minute → 2DIGIT

second → 2DIGIT



2DIGIT → DIGIT DIGIT

DIGIT  → “0”|”1”|”2”|”3”|”4”|”5”|”6”|”7”|”8”|”9“
```

```
time   → hour ":" minute
       | hour ":" minute ":" second

hour   → 2DIGIT

minute → 2DIGIT

second → 2DIGIT



2DIGIT → DIGIT DIGIT

DIGIT  → “0”|”1”|”2”|”3”|”4”|”5”|”6”|”7”|”8”|”9“
```

Pojmovi (neterminali): time, hour, minute, second, 2DIGIT, DIGIT
Simboli (terminali): “0”, “1”, “2”..., “9”, “:”

Terminalno završno; Nema dalje produkcija.
Terminali se ne mogu naći sa leve strane.

```
time   → hour ":" minute
       | hour ":" minute ":" second

hour   → 2DIGIT

minute → 2DIGIT

second → 2DIGIT


2DIGIT → DIGIT DIGIT

DIGIT  → "0"|"1"|"2"|"3"|"4"|"5"|"6"|"7"|"8"|"9"
```

2DIGIT, DIGIT - mogli bi biti tokeni, prepoznati u leksičkoj analizi, na osnovu druge gramatike (gramatika leksike, regularni izrazi). Tada bi oni bili terminali za tu gramatiku.

Skener – azbuka: karakteri
Parser – azbuka: tokeni

```
text
   → sentence
   → text sentence

sentence
   → words dot

words
   → capital_word
   → words word
   → words capital_word

word
   → small_letter
   → word small_letter

capital_word
   → capital_letter
   → capital_word small_letter

dot
   → "."
```

Primer jezika i odgovarajuća gramatika za opis teksta.

- Šta su tokeni (simboli, terminali)?
- Šta su pojmovi?

```
capital_letter
   →  "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H"
    | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P"
    | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X"
    | "Y" | "Z"
small_letter
   →  "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h"
    | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p"
    | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x"
    | "y" | "z"
```

```
text
   → sentence
   → text sentence

sentence
   → words dot

words
   → capital_word
   → words word
   → words capital_word

word
   → small_letter
   → word small_letter

capital_word
   → capital_letter
   → capital_word small_letter

dot
   → "."
```

Primer jezika i odgovarajuća gramatika za opis teksta.

- Šta su ispravni ulazi?

```
capital_letter
   →  "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H"
    | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P"
    | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X"
    | "Y" | "Z"
small_letter
   →  "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h"
    | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p"
    | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x"
    | "y" | "z"
```

```
text
   → sentence
   → text sentence

sentence
   → words dot

words
   → capital_word
   → words word
   → words capital_word

word
   → small_letter
   → word small_letter

capital_word
   → capital_letter
   → capital_word small_letter

dot
   → "."
```

Primer jezika i odgovarajuća gramatika za opis teksta.

- Šta su ispravni ulazi?

Danas pada kiša.	OK
pada.	ERR
kiša pada	ERR

```
capital_letter
   →  "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H"
    | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P"
    | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X"
    | "Y" | "Z"
small_letter
   →  "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h"
    | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p"
    | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x"
    | "y" | "z"
```

```
text
   → sentence
   → text sentence

sentence
   → words dot

words
   → capital_word
   → words word
   → words capital_word

word
   → small_letter
   → word small_letter

capital_word
   → capital_letter
   → capital_word small_letter

dot
   → "."
```

Primer jezika i odgovarajuća gramatika za opis teksta.

Šta bi još mogli biti tokeni?

```
capital_letter
   →  "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H"
    | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P"
    | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X"
    | "Y" | "Z"
small_letter
   →  "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h"
    | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p"
    | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x"
    | "y" | "z"
```

Proširene BNF forme

Dodaju se oznake:

- […] – ponavljanje sadržaja 0 ili 1 put
- { … } – ponavljanje sadržaja 0 ili više puta
- (…) – grupisanje

- nekad: “,” konkatenacija, “;” kraj
- nekad: i *, +, ?

```
text
   → sentence
   → text sentence

sentence
   → words dot

words
   → capital_word
   → words word
   → words capital_word

word
   → small_letter
   → word small_letter

capital_word
   → capital_letter
   → capital_word small_letter

dot
   → "."
```

```
capital_letter
   →  "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H"
    | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P"
    | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X"
    | "Y" | "Z"
small_letter
   →  "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h"
    | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p"
    | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x"
    | "y" | "z"
```

```
text
   → sentence { sentence }
sentence
   → words dot
words
   → capital_word { ( word | capital_word ) }


word
   → small_letter { small_letter }
capital_word
   → capital_letter { small_letter }
dot
   → "."
```

Kako da napišemo parser za jezik definisan ovom gramatikom?

```
text
   → sentence
   → text sentence

sentence
   → words dot

words
   → capital_word
   → words word
   → words capital_word
```

pp: capital_word, word i dot su terminali

Kako da napišemo parser za jezik definisan ovom gramatikom?

```
text
   → sentence
   → text sentence

sentence
   → words dot

words
   → capital_word
   → words word
   → words capital_word
```

pp: capital_word, word i dot su terminali

Ovo je tekst.

`w = capital_word word word dot`

text
⇒ *sentence*
⇒ *words* dot
⇒ *words* word dot
⇒ words word word dot
⇒ capital_word word word dot

text $\Rightarrow^*$ w

```
text
   → sentence
   → text sentence

sentence
   → words dot

words
   → capital_word
   → words word
   → words capital_word
```

pp: capital_word, word i dot su terminali

Ovo je tekst.

`w = capital_word word word dot`

text
⇒ *sentence*
⇒ *words* dot
⇒ *words* word dot
⇒ words word word dot
⇒ capital_word word word dot

text $\Rightarrow^*$ w

Parser - algoritam kojim možemo za bilo koji ulaz da kažemo:

- našli smo izvođenje prateći pravila (parsiranje je uspešno) ILI
- ne postoji nijedno izvođenje (sintaksna greška)